CO1_AT1_AI

Analytical Problem Solving

1. Water Jug Problem (4-gallon and 3-gallon jugs)

Problem formulation: State = (x, y), where x is water in the 4-gallon jug and y is water in the 3-gallon jug. Initial state: (0,0). Goal state: any state with x = 2. Operators: fill a jug, empty a jug, or pour from one jug to the other until the source is empty or destination is full.

Step	Action	State (4-gal, 3-gal)
0	Start; both empty	(0,0)
1	Fill 3-gallon jug	(0,3)
2	Pour 3-gallon into 4-gallon jug	(3,0)
3	Fill 3-gallon jug again	(3,3)
4	Pour into 4-gallon jug; only 1 gallon fits, leaving 2	(4,2)
5	Empty 4-gallon jug	(0,2)
6	Pour remaining 2 gallons into 4-gallon jug	(2,0)

Result and verification: The final state is (2,0), so the 4-gallon jug contains exactly 2 gallons. Every action follows the allowed operators.

2. Mars Rover as an Intelligent Agent

i. Percepts: Camera images, proximity/distance sensors, wheel encoders, estimated location, temperature, and soil/rock readings.

ii. Environment: Martian terrain containing rocks, slopes, dust, craters, changing light/weather, scientific targets, and delayed communication with Earth.

iii. Actions: Move forward/backward, turn, stop, avoid obstacles, capture images, operate scientific instruments, analyse/collect samples, and transmit data.

iv. Performance measures: Scientific usefulness of data, successful target/sample analysis, navigation accuracy, energy efficiency, time, safety, and mission completion.

v. Agent architecture: A model-based, goal-based agent with utility-based decision-making is suitable. It maintains an internal map/state, plans toward science goals, and weighs risk, energy, and scientific value before acting. Learning can improve future decisions from observations.

3. 8-Queens Problem: Search-Space Formulation

Initial state: Empty 8×8 board, unless the question provides a partial board. **State:** Locations of queens placed so far. **Actions:** Place a queen in a safe square in the next column. **Goal test:** Eight queens are placed with no two in the same row, column, or diagonal. **Path cost:** One per placement (or zero if only solution existence matters).

Method: Use depth-first search with backtracking. Place one queen per column. For each row, check that no previous queen shares its row or diagonal. If no safe row exists, backtrack to the previous column and move that queen. Stop when all 8 queens are placed.

Conflict test: Queens at (r■,c■) and (r■,c■) conflict if r■=r■ or |r■–r■|=|c■–c■|. One queen per column is guaranteed by the placement order.

> The supplied picture's initial board configuration is not legible enough to verify specific pre-placed queens; the formulation and algorithm above are the general search-space solution.

4. OLA Cab Booking: Problem-Solving Agent

Agent type: A utility-based agent is suitable because it can balance cab type, availability, fare, waiting time, travel time, and comfort. It should book only after the user confirms.

Pseudocode

```
START
Read pickup, destination, cab_type, comfort_preference
Get available cabs matching cab_type
FOR each cab: estimate fare, arrival time, trip time and comfort
Reject unavailable cabs and those violating hard preferences
Score remaining cabs using user preferences
Recommend the cab with the best score; show fare/time estimate
IF customer confirms: book cab and send trip details
```

```
ELSE: allow customer to change preferences
END
```

Explanation: The agent perceives user input and cab availability, evaluates alternatives, selects the most suitable option, and performs the booking after confirmation. Hard constraints are checked before preference scoring.

5. Uniform Cost Search (UCS): Least-Cost Route

UCS always expands the frontier path with the smallest cumulative cost g(n). The graph edge costs are S–A=1, S–G=12, A–B=3, A–C=1, B–D=3, C–D=1, C–G=2 and D–G=3.

Step	Expanded node	Frontier after expansion (cost)
1	S (0)	A(1), G(12)
2	A (1)	C(2), B(4), G(12)
3	C (2)	D(3), G(4), B(4), G(12)
4	D (3)	G(4), B(4), G(6), G(12)
5	G (4)	Goal selected; stop

Cost comparison: S→G = 12; S→A→B→D→G = 1+3+3+3 = 10; S→A→C→D→G = 1+1+1+3 = 6; S→A→C→G = 1+1+2 = 4.

Final answer: Optimal route is **S** → **A** → **C** → **G**; total cost = **4**. UCS is optimal when edge costs are non-negative. The route is verified by summing its edge costs: 1+1+2=4.

CO1_AT2_AI

Constraint-Based Problem Solving

1. Assigning Three Doctors to Three Shifts Using Backtracking

Variables: D1, D2, D3. **Domain:** Morning (M), Afternoon (A), Night (N). **Constraints:** D1≠N; D2 must work before D3; each shift has one doctor; D3≠M; each doctor has exactly one shift.

Backtracking procedure: Assign one doctor at a time, check constraints immediately, and backtrack if any rule is violated.

Step	Trial assignment	Constraint check
1	D1 = M	Valid; D1 is not Night.
2	D2 = A	Possible; leaves Night for D3.
3	D3 = N	Valid; D2 is before D3, D3 is not Morning, and shifts are distinct.
4	Check all constraints	All given constraints are satisfied.

Solution: Morning—D1; Afternoon—D2; Night—D3. This is one valid schedule under the stated constraints.

2. Robot in a 5×5 Grid: BFS and Manhattan Distance

State representation: Each unblocked cell is (row,column). Start = S; goal = G. Legal actions: Up, Down, Left, Right. Each move costs 1, and blocked cells cannot be entered.

BFS algorithm: Put S in a FIFO queue and mark it visited. Repeatedly remove the front cell. If it is G, reconstruct the path using parent pointers. Otherwise, examine the four neighbours; enqueue each neighbour that is inside the grid, unblocked, and unvisited. Mark it visited and store its parent when enqueued. If the queue becomes empty, no path exists.

Manhattan heuristic: h(n)=|row(n)–row(G)|+|column(n)–column(G)|. It estimates horizontal plus vertical moves to G. It ignores obstacles, so it is a lower bound on the true path length. BFS guarantees a shortest path when every move has equal cost; A* can use Manhattan distance to guide informed search.

Pseudocode

```
queue ← [S]; visited ← {S}; parent[S] ← null
WHILE queue is not empty:
  current ← remove_front(queue)
  IF current = G: return path reconstructed from parent links
  FOR neighbour in [Up, Down, Left, Right]:
    IF inside grid AND not blocked AND not visited:
      visited.add(neighbour); parent[neighbour] ← current
      add neighbour to back of queue
RETURN “No path exists”
```

The exact S/G coordinates and obstacle cells are not sufficiently clear in the supplied image, so a specific cell-by-cell route cannot be confirmed. The algorithm above is the correct general procedure.

3. Autonomous Rescue Robot in a Disaster-Affected Building

a) Constraint analysis

• **Movement:** Up, down, left and right only; diagonal movement is forbidden.
• **Obstacles:** Blocked rooms/cells cannot be entered.
• **Limited sensing:** Only adjacent cells are observed, so the complete map is not initially known.
• **Costs:** Normal cell entry costs 1. If a risky cell adds 2 extra cost, its total entry cost is 3.
• **Goal and safety:** Reach G while minimizing total cost and avoiding unnecessary risk.
• **Changing knowledge:** Update the map from each observation and replan when a route becomes invalid.

b) Solution using Uniform Cost Search and online exploration

1. Set cost(S)=0 and insert S into a priority queue.
2. Sense adjacent cells and update the known map.
3. Generate only legal, unblocked neighbours.
4. Add cost 1 for a normal cell or 3 for a risky cell (1+2 extra).
5. If a cheaper path to a cell is found, update its cost and parent.
6. Expand the frontier cell with the lowest cumulative cost.
7. When G is reached, reconstruct the least-cost path.
8. Because the map is partially known, continue sensing and replan if new information reveals obstacles or a safer/cheaper route.

c) AI concepts

• **Intelligent agent:** Senses the environment, selects an action, and acts toward the rescue goal.
• **Rationality:** Chooses the action with the best expected safety and path cost given current information.
• **Environment:** Partially observable, sequential, discrete/grid-based, dynamic during exploration, and potentially stochastic if sensing or movement is uncertain.

d) Justification

Uniform Cost Search is appropriate because normal and risky cells have different costs. It expands the lowest cumulative-cost path first and finds a minimum-cost route when costs are non-negative and the map is known. Since sensing is limited, the robot must combine UCS with repeated observation and replanning. A* may reduce search effort when a sufficiently complete map is available and an admissible heuristic is used.

Conclusion

Backtracking finds assignments that satisfy all constraints. BFS finds shortest paths when moves have equal cost, while UCS is suitable for routes with different costs. In a partially observable rescue environment, sensing and replanning are essential.

1. An intelligent agent is a system that observers its environ-ment through sensors and performs actions using actua-tors to acheive a particular objective

In a smart home intelligent agents control lighting tempe-rature, and security based on user behaviour and environmental conditions.

(i) Simple Reflex Agent.

A simple reflex agent takes action based only on the current input using condition - action rules.

Ex:

If motion is detected the light turnes on.

If the temperature is high, the fan turns on.

If Unauthorized entry is detected, the alarm activities.

It is that and simple but cannot remember past information or learn user perforences.

(ii) Model - Based Agent

A model - based agent maintains an internal state of the environment room using current and previous information.

Ex: It remembers room occupancy and prevents lights from turning off when a person is tempo-rarity outside the sensor's range.

It handles changing environment effectively but requires additional memory

B. (iii) Goal - Based Agent

A Goal - based selects actions to achelve a specific objective.

Ex : It adjusts the air conditioner to maintain the desired temperature and ensure that doors and loked. when nobody is home.

It supports planning but may require more computation.

(v) Utility - Based Agent

A Utility - Based agent evalrates different outcomes and selects the most beneficial action based on factors such as comfort, energy consuption and security.

Ex : It adjust the air conditioner to maintain comfort. while reducing electrity consuption

It balance multiple objectives but requires a suitable utility function.

Most effective Approach :

The model - based agent remember environmental conditions, the goal - based agent works to wards comfort and security and comfort and the simple reflex agent reports quickly to emergencies

conclusion :

A hyboria intelligent agent provids comfort energy efficency and security while adaping to user preferences and environmental changes.

Uninformed search strategies explore possible paths without perior knowledge of the maze's layout. Uniform cost search (UCS) Interative Deepening search (IDS) can help a robot find the exit.

(i) Uniform cost search (UCS

UCS exparels the node with the lowest travel path cost using a priority queve.

working :

* Start from the Initial position.
* Explore available paths and calculate their costs.
* select the node with the lowest path cost.
* Repeat when the exit is reached.

Advandages :

* Finds the least-cost path when step costs are non-negative and suitable conditions hold.
* Handles different path costs effectively

Disadvandages :

* Requires more memory
* May explore no It regenerates upper-level nodes repeatdly (small overhead) ignore step costs and needs cycle checlip.

Relation to agent Types :

Simple reflex : fails, as it has no memory and loop in the maze

model - based :

needed since the maze is unknown It nudds a ma and remembers visited cells.

Goal based :

Searches for a Path to the exit.

Utility based :

Useful when moves differen in cost (energy, time, risk)

Recommended comtrination :

a moodel based goal based apent with IDS which is memory efficient and finds with the shallowest exit. It more costs vary, use a Utility bassed agent with ucs for the optimal Loost-cos Path acepting hiper memory use.